NATURAL LANGUAGE PROCESSING — FALL 2026

Assignment 1: Text Classification with Naive Bayes

Student ID: 2305467 | GitHub: FekyBaz | Repository: github.com/FekyBaz/nlp-naive-bayes-assignment6

This report summarizes a bag-of-words text classifier built for a four-class sentence dataset. A Multinomial Naive Bayes model is implemented from scratch with Laplace smoothing and log-space scoring, then compared with scikit-learn's `MultinomialNB` on exactly the same 68/17 split. Both models reach **100.00% training accuracy** and **82.35% test accuracy** (14/17 correct), and make **identical predictions on both splits** (training agreement 100%, test agreement 100%). The small test set and the gap between training and test accuracy call for cautious interpretation.

1. Dataset

The repository includes `synthetic_text_data.csv` with 85 English sentences and four labels: Entertainment 19, Politics 21, Sports 15, Technology 30. The file has no missing values, no duplicate texts, and no empty sentences. Sentences average 12 alphabetic tokens (range 8–16). The classes are imbalanced: Technology is the largest class and Sports the smallest. Figure 1 (on page 2) shows the class distribution and the most frequent training words. As expected without stopword removal, function words such as “the”, “are”, and “to” dominate the frequency ranking.

2. Preprocessing, split, and vocabulary

Each sentence is lowercased and tokenized with the regular expression `[a-z]+`, which removes punctuation, digits, and non-ASCII characters. Rows that become empty after tokenization are dropped (none did). The cleaned data is split with `test_size=0.2`, `random_state=42`, and stratification, producing **68 training** and **17 test** sentences. The vocabulary is built from the training tokens only: **391 distinct words**. Test-only words are safely ignored, giving **69 unseen test types** and **0 test documents with no in-vocabulary tokens**. No test sentence is used for the vocabulary, word frequencies, or word clouds.

3. Models

Manual Multinomial Naive Bayes. For each class *c*, the prior and smoothed word likelihood are estimated from the training counts:

$$P(c) = N_c / N \qquad P(w \mid c) = (\text{count}(w, c) + \alpha) / (\Sigma_w \text{count}(w, c) + \alpha |V|)$$

$$\hat{y} = \text{argmax}_c [\log P(c) + \Sigma_w x_w \log P(w \mid c)]$$

with `α = 1` (Laplace smoothing) and `|V| = 391`. The implementation follows the section's four-step formulation (priors, vocabulary/denominators, likelihoods, class scores) and builds BoW vectors with `Counter`. Counts are weighted by the BoW vector and summed in log space to avoid numerical underflow. Unknown words contribute nothing because they are absent from the vocabulary. **scikit-learn baseline.** `MultinomialNB(alpha=1.0)` is trained on the same matrices and labels. Because both models use empirical priors and the same smoothed multinomial likelihood, identical predictions are expected.

4. Results

Model	Train accuracy	Test accuracy	Correct test predictions
Manual Multinomial NB	1.0000	0.8235	14/17
scikit-learn MultinomialNB	1.0000	0.8235	14/17

Both models produce the same predictions (training agreement **100%**, test agreement **100%**). The per-class test metrics are therefore identical for the two implementations:

Class	Precision	Recall	F1-score	Support
Entertainment	1.0000	0.7500	0.8571	4
Politics	0.8000	1.0000	0.8889	4
Sports	1.0000	0.6667	0.8000	3
Technology	0.7143	0.8333	0.7692	6
Macro average	0.8786	0.8125	0.8288	17
Weighted average	0.8521	0.8235	0.8235	17

Politics is the only class with perfect recall (1.00); Technology has the lowest F1-score (0.77) because it is both the largest class and the source or target of most confusions. The macro F1 is 0.8288 and the weighted F1 is 0.8235. Perfect training accuracy versus 82.35% test accuracy is consistent with overfitting, but with only 17 test sentences a single error moves accuracy by about 5.9 percentage points, so the estimate is uncertain.

5. Visual diagnostics and error analysis

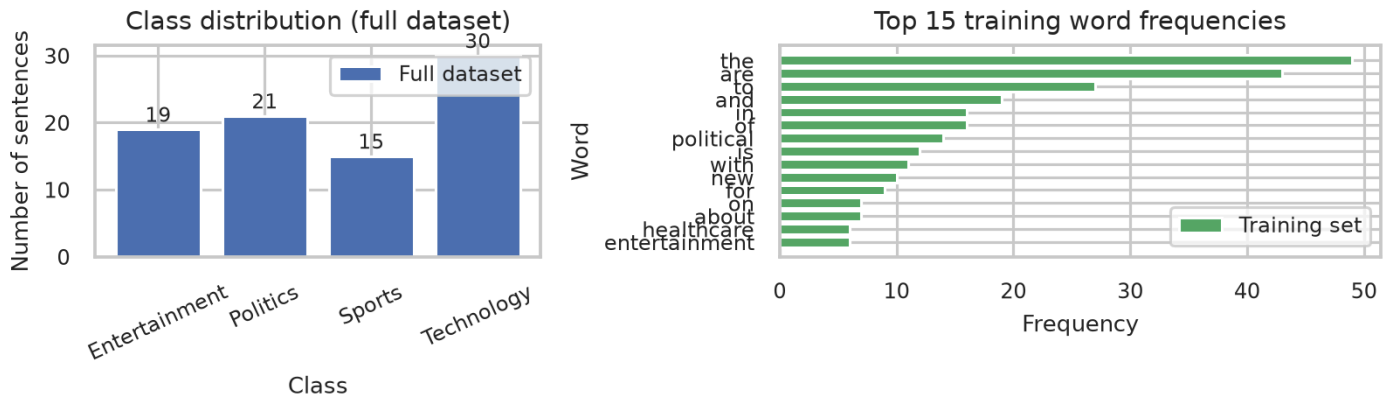


Figure 1. Left: class distribution in the full dataset (Entertainment 19, Politics 21, Sports 15, Technology 30). Right: the 15 most frequent tokens in the training split. Frequent function words dominate because the assignment's preprocessing does not remove stopwords.

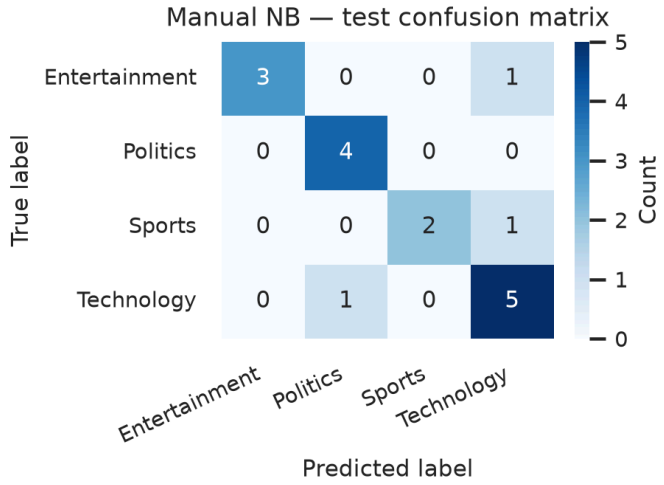


Figure 2. Test-set confusion matrix for the manual model. The scikit-learn matrix is identical.

What the three errors show

- **Sports → Technology**: *The summer Olympics are fast approaching, and athletes are training harder than ever.*
- **Entertainment → Technology**: *Digital content creators are leading the charge in shaping modern entertainment culture.*
- **Technology → Politics**: *Artificial intelligence could soon have a huge impact on the job market.*

All errors are single-label confusions rather than a class collapsing into another. They reflect the main limitation of a bag-of-words model: individual words such as "digital", "training", "market", and "impact" carry overlapping associations across topics, and word order or multi-word context is discarded. The model nevertheless recovers Politics perfectly and classifies 14 of 17 sentences correctly.

6. Discussion and limitations

The manual implementation matches scikit-learn exactly, which validates the count aggregation, prior calculation, Laplace smoothing, and log-space scoring. The main performance limitation is the data: 85 short synthetic sentences, noticeable class imbalance, and a 17-sentence test set. The model also uses a simple BoW representation with no stopword removal, stemming, n-grams, or hyperparameter tuning. These choices keep the assignment scope controlled but mean the reported accuracy should not be read as a generalisation estimate. Cross-validation on a larger, more balanced corpus would give a more reliable comparison. The test set was used only for final evaluation; all vocabulary and word-based exploratory analysis came from the training split.

7. Reproducibility

Environment: Python 3.11+ (verified on 3.12 locally and 3.11 in GitHub Actions), numpy, pandas, matplotlib, seaborn, scikit-learn, wordcloud (see `requirements.txt`).
Notebook: `assignment_1.ipynb` (source) and `assignment_1_executed.ipynb` (saved outputs).
Execute: `python -m jupyter nbconvert --to notebook --execute assignment_1.ipynb --output assignment_1_executed.ipynb`
Validate: `python scripts/validate_notebook.py assignment_1_executed.ipynb`
Report build: `python report/generate_report.py`